

avito.tech 

ПОВЕДЕНЧЕСКИЕ ПАТТЕРНЫ

Афанасьев Юра

MEMENTO

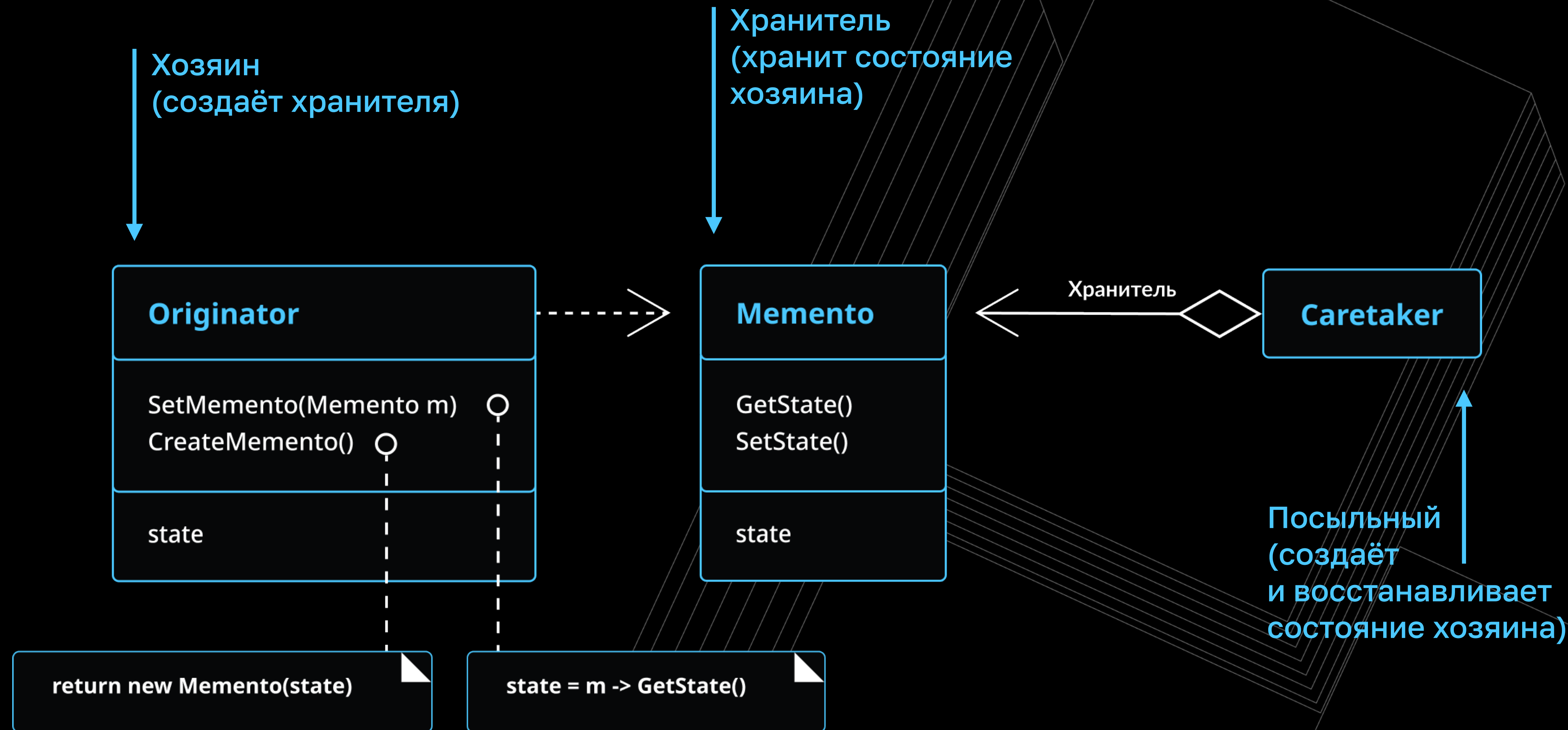


ХРАНИТЕЛЬ

Не нарушая инкапсуляции,
фиксирует и выносит за пределы
объекта его внутреннее состояние
так, чтобы позднее можно было
восстановить в нем объект

Решаемая задача

- Фиксирует внутреннее состояние объекта вне класса
- Запрещает доступ к состоянию, кроме исходного класса
- Позволяет сохранять и восстанавливать любое состояние объекта



Хозяин

```
class Originator
{
    private string $state;

    public function setState(string $state): void
    {
        $this->state = $state;
        echo 'Установлено новое состояние ' . $this->state . PHP_EOL;
    }

    public function createMemento(): Memento
    {
        echo 'Сохраняем состояние ' . $this->state . PHP_EOL;
        return new Memento($this->state);
    }

    public function setMemento(Memento $memento): void
    {
        $this->state = $memento->getState();
        echo 'Изменилось состояние на ' . $memento->getState() . PHP_EOL;
    }
}
```

Таких методов
может быть много
и каждый решает
свою задачу

Изменяет
состояние
хозяина

Запоминает
состояние

Меняет текущее
состояние
на новое

Инфраструктурные
методы
для реализации
паттерна

Хранитель

```
class Memento
{
    private string $state;

    public function __construct(string $state)
    {
        $this->state = $state;
    }

    public function getState(): string
    {
        return $this->state;
    }
}
```

Если у хозяина много
свойств, то передаём
их все

Хранит полное
состояние хозяина

Посыльный
(прослойка для
манипулирования)

Набор методов
зависит
от решаемых задач

```
class Caretaker
{
```

```
    private Originator $originator;
    /** @var Memento[] */
    private array $mementos;
```

Оперирует
исключительно
ХОЗЯИНОМ

```
    public function __construct(Originator $originator)
    {
        $this->originator = $originator;
    }
```

Создаёт снимок
ХОЗЯИНА

```
    public function backup(): void
    {
        $this->mementos[] = $this->originator->createMemento();
    }
```

Это позволит
откатывать их,
просматривать и т.д.

Откатывает
состояние хозяина

```
public function undo(): void
{
    if (!count($this->mementos)) {
        return;
    }

    array_pop($this->mementos);
    $memento = end($this->mementos);
    $this->originator->setState($memento->getState());
}

public function showStateHistory(): void
{
    echo 'Выводим историю состояний:' . PHP_EOL;
    foreach ($this->mementos as $key => $memento) {
        echo $key . '. ' . $memento->getState() . PHP_EOL;
    }
}
}
```

Возвращает все
состояния хозяина

Меняем состояние
и сохраняем его

```
function testMemento()  
{
```

```
    $originator = new Originator();  
    $caretaker = new Caretaker($originator);
```

```
    $originator->setState('first');  
    $caretaker->backup();
```

```
    echo PHP_EOL;
```

```
    $originator->setState('second');  
    $caretaker->backup();
```

```
    echo PHP_EOL;
```

```
    $originator->setState('third');  
    $caretaker->backup();
```

Создаём хозяина

Передаём посылному

0. first
1. second
2. third

Откатываем
последнее
состояние

0. first
1. second
2. fourth

```
echo PHP_EOL;

$caretaker->showStateHistory();

echo PHP_EOL;

$caretaker->undo();

echo PHP_EOL;

$originator->setState('fourth');
$caretaker->backup();

echo PHP_EOL;

$caretaker->showStateHistory();
}
```

Добавляем состояние
и сохраняем его

Плюсы

- Сохраняются границы инкапсуляции
- Упрощает реализацию исходного объекта

Минусы



- Издержки памяти при хранении большого объёма данных
- Сложно гарантировать доступ к хранителю только хозяину (в динамических языках)

Фиксирует
внутреннее
состояние
объекта

TEMPLATE METHOD

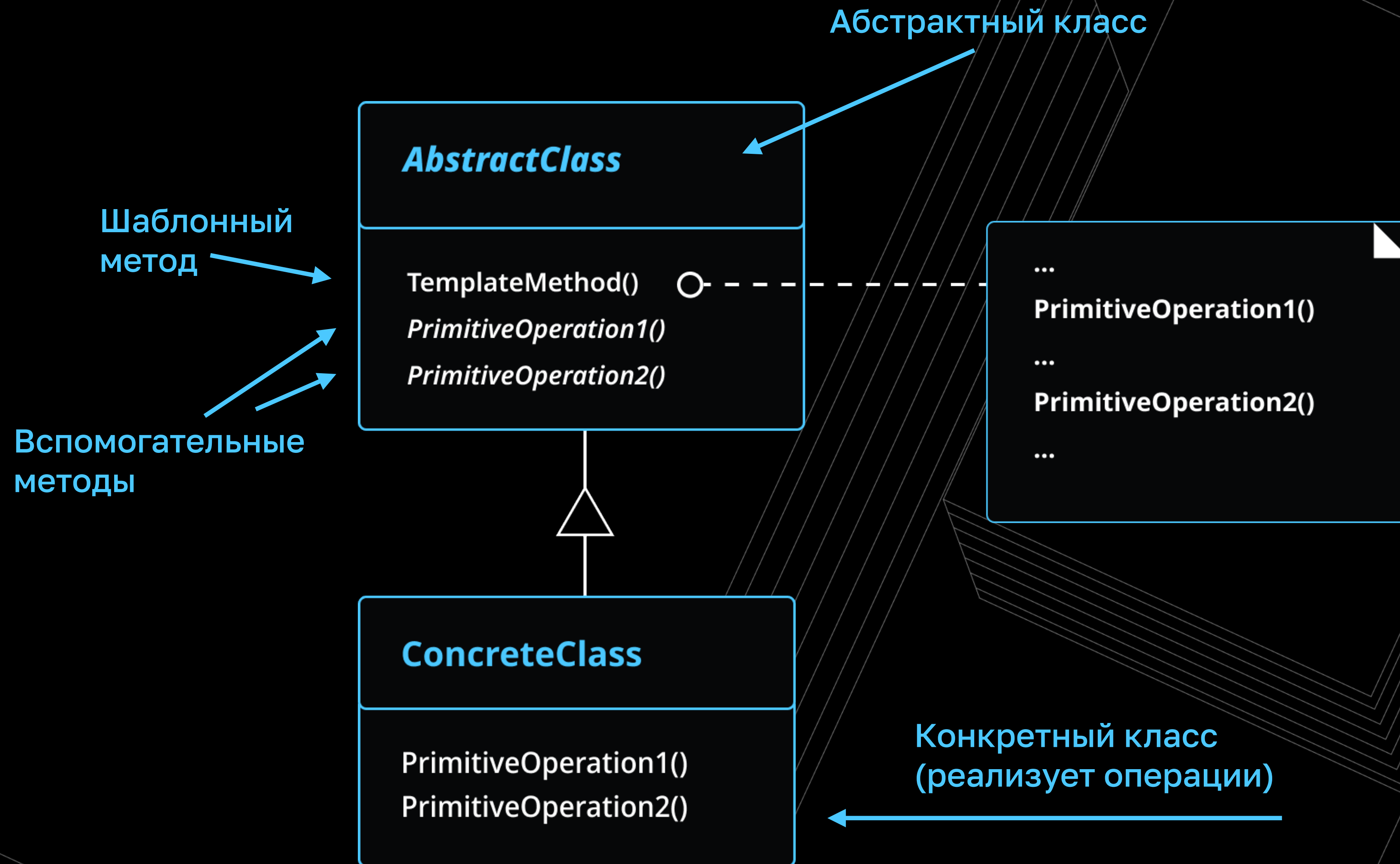


ШАБЛОННЫЙ МЕТОД

Определяет основу алгоритма
и позволяет подклассам
переопределить некоторые шаги
алгоритма, не изменяя
его структуру в целом

Решаемая задача

- Выделяет общее поведение для избежания дублирования кода (удовлетворяет DRY)
- Управляет расширяемостью подклассов



Абстрактный
класс

Финальный

Здесь
дефолтный
результат

Невозможно
переопределить

```
abstract class AbstractClass
```

```
{
```

```
    final public function templateMethod()
```

```
{
```

```
        $this->operation1();
```

```
        $this->operation2();
```

```
        $this->operation3();
```

```
        $this->operation4();
```

```
}
```

```
    protected function operation1()
```

```
{
```

```
        echo 'Операция 1. Может быть переопределена.' . PHP_EOL;
```

```
}
```

```
    abstract protected function operation2();
```

```
    protected function operation3()
```

```
{
```

```
        return true;
```

```
}
```

```
    private function operation4()
```

```
{
```

```
        echo 'Операция 4. Нельзя переопределить.' . PHP_EOL;
```

```
}
```

```
}
```

Шаблонный
метод

Вспомогательные
методы

Метод возможно
переопределить

Необходимо
реализовать

Операция
зацепки

Конкретный
класс

Расширяет
базовый класс

```
class ConcreteClass extends AbstractClass
{
    protected function operation1()
    {
        echo 'Новая операция 1' . PHP_EOL;
    }

    protected function operation2()
    {
        echo 'Новая операция 2' . PHP_EOL;
    }

    protected function operation3()
    {
        echo 'Новая операция 3' . PHP_EOL;
    }
}

// class AnotherConcreteClass extends Main { }
```

На каждую
реализацию методов
создаётся свой
конкретный класс

В зависимости
от класса
получаем разные
результаты

```
function testTemplateMethod()  
{  
    (new ConcreteClass())->templateMethod();  
  
    // (new AnotherConcreteClass())->templateMethod();  
}
```

Шаблонный метод

Плюсы

- Помогает избежать дублирования кода путём выделения общего поведения для всех подклассов
- Реализует принцип открытости/закрытости

Минусы

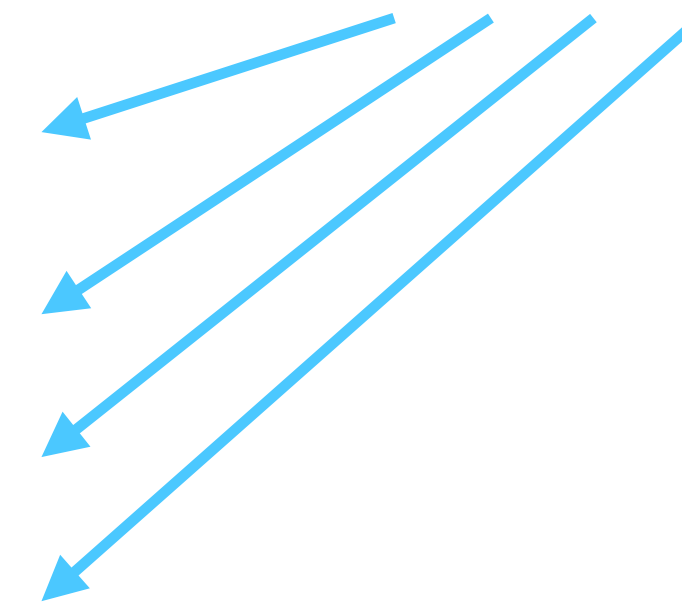


- Чем больше переопределяется методов в наследнике, тем дольше его реализовывать
- Подходит для небольших алгоритмов

```
abstract class Main
{
    final public function templateMethod()
    {
        $this->operation1();
        $this->operation2();
        $this->operation3();
        $this->operation4();
        $this->operation5();
        $this->operation6();
        $this->operation7();
        $this->operation8();
    }
}
```

```
abstract protected function operation1();
abstract protected function operation2();
abstract protected function operation3();
abstract protected function operation4();
abstract protected function operation5();
abstract protected function operation6();
abstract protected function operation7();
abstract protected function operation8();
}
```

Слишком много
методов для реализации



Шаблон вызывает операции

- Конкретные операции из классов наследников
- Конкретные операции из абстрактного (текущего) класса
- Фабричные методы (паттерн Фабричный метод)
- Операции зацепки (hook), реализующие поведение по умолчанию

```
abstract class Main
```

```
{
```

```
    final public function templateMethod()
```

```
    {
```

```
        $this->operation1();
```

```
        $this->operation2();
```

```
        $this->operation3();
```

```
        $this->operation4();
```

```
    }
```

```
    protected function operation1()
```

```
    {
```

```
        echo 'Операция 1. Может быть переопределена.';
```

```
    }
```

```
    abstract protected function operation2();
```

```
    protected function operation3()
```

```
    {
```

```
        return true;
```

```
    }
```

```
    protected function operation4()
```

```
    {
```

```
        return new stdClass();
```

```
    }
```

```
}
```

Шаблонный
метод

Конкретная операция
из текущего класса

Конкретная операция,
реализуемая наследником

Операция
зацепки

Здесь дефолтный
результат

Фабричный
метод

Применяйте инкапсуляцию верно

- public — вызываемый шаблонный метод
- protected — методы, можно переопределять в наследнике
- private — невозможно переопределить

Особенности реализации

- Одна из реализаций Голливудского принципа (родительский класс вызывает методы подкласса)
- Количество переопределяемых методов должно быть немного

Описывает алгоритм
действий,
реализуемый через
наследование
и абстрактные методы

avito.tech

Смотрите все серии на сайте
<https://avito.tech/patterns>

